ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Ukládání a serializace dat

Základy programování

Matouš Cejnek
U12110, FS, ČVUT v Praze

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Co si dnes odnesete?

- Vysvětlit serializaci a deserializaci.
- Rozlišit datový model, formát a soubor.
- Uložit a načíst data pomocí modulů `json` a `csv`.
- Předpovědět, které datové typy se při převodu zachovají.
- Zvolit formát podle tvaru dat, příjemce a důvěryhodnosti zdroje.
- Bezpečně posoudit použití binárního formátu `pickle`.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Na co navazujeme?

- Slovníky a seznamy nesou strukturovaná data v paměti.
- `with open(...)` bezpečně spravuje otevřený soubor.
- UTF-8 převádí text na bajty a zpět.
- Moduly standardní knihovny importujeme a používáme přes jejich API.
- Očekávatelné chyby umíme zachytit pomocí `try`/`except`.

Z domácí přípravy

Proč se po načtení knih z CSV změnily `year` a `available` na řetězce?

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

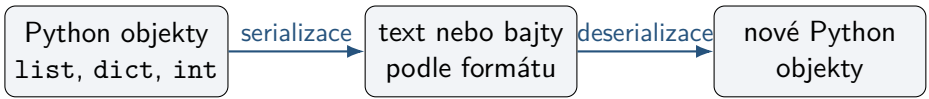
Objekt v paměti po ukončení programu zmizí

- Python pracuje s objekty: seznamy, slovníky, čísly a instancemi tříd.
- Soubor nebo síť přenáší pouze posloupnost znaků či bajtů.
- Příští spuštění programu potřebuje strukturu z této posloupnosti obnovit.

Otázka

Jak zapíšeme strukturu dat tak, aby její význam znal i budoucí program?

Serializace je převod reprezentace



Deserializace nevzkřísí původní objekt. Vytvoří nové objekty z uložené reprezentace.

Formát není totéž co místo uložení

Datový model Jaké hodnoty a vztahy potřebujeme vyjádřit?

Formát Jak budou data zapsána? JSON, CSV, binární formát.

Médium Kde reprezentace leží? Soubor, databáze, síťová zpráva.

Kódování Jak se text převede na bajty? Typicky UTF-8.

Přípona nestačí

Soubor pojmenovaný `books.json` nemusí obsahovat platný JSON ani očekávaná data.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Formát vybíráme podle budoucího čtenáře

1. Je struktura plochá tabulka, nebo obsahuje vnoření?
2. Musí data číst člověk, tabulkový procesor, Python, nebo více jazyků?
3. Potřebujeme zachovat čísla, logické hodnoty a chybějící hodnotu?
4. Jak velká data budou a musíme je zpracovávat postupně?
5. Je zdroj důvěryhodný a jak dlouho mají data vydržet?

Nejdřív požadavky, potom formát.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

JSON má malý přenositelný datový model

objekt dvojice klíč–hodnota; klíč je řetězec,

pole uspořádaná posloupnost hodnot,

skalární hodnoty řetězec, číslo, `true`, `false`, `null`.

JSON není Python

Podobná syntaxe neznamená stejné typy: JSON nezná n-tici, množinu, datum ani vlastní třídu.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Základní mapování typů

Python	JSON	po načtení
`dict` se stringovými klíči	object	`dict`
`list`	array	`list`
`tuple`	array	`list`
`str`	string	`str`
`int`, `float`	number	číslo
`True`, `False`	true, false	`bool`
`None`	null	`None`

„Lze zapsat“ neznamená vždy „vrátí se přesně stejný typ“.

Ne každý Python objekt projde beze změny

```
import json

data = {
    "position": (10, 20),
    "tags": {"python", "data"},
}

text = json.dumps(data)  # TypeError kvuli set
```

- N-tice by se po načtení vrátila jako seznam.
- Množina není součástí datového modelu JSON.
- Datum či vlastní objekt musíme převést podle vlastní dohody.

json.dump zapisuje do otevřeného souboru

```
import json

with open("books.json", "w", encoding="utf-8") as file:
    json.dump(
        books,
        file,
        ensure_ascii=False,
        indent=2,
    )
```

- Režim `"w"` původní obsah přepíše.
- `json.dump` očekává textový soubor, ne binární.

`json.load` obnoví data ze souboru

```
import json

with open("books.json", encoding="utf-8") as file:
    loaded_books = json.load(file)

print(type(loaded_books))              # list
print(type(loaded_books[0]["year"])) # int
```

Načtený seznam je nový objekt. Změna `loaded_books` sama o sobě soubor nezmění.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Dvě souborové a dvě řetězcové funkce

Funkce	**Vstup nebo výstup**	**Použití**
`json.dump(data, file)`	otevřený soubor	zápis
`json.load(file)`	otevřený soubor	čtení
`json.dumps(data)`	vrací `str`	text, síť, log
`json.loads(text)`	přijímá `str`	text v paměti

Písmeno `s` můžeme číst jako *string*.

Čitelnost mění zápis, ne význam dat

```
compact = json.dumps(books, ensure_ascii=False)
readable = json.dumps(
    books,
    ensure_ascii=False,
    indent=2,
)
```

`ensure_ascii=False` ponechá české znaky čitelné.

`indent=2` přidá odsazení a nové řádky.

Čitelnější soubor bývá větší; po načtení však dostaneme stejná data.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Poškozený JSON je očekávatelná chyba vstupu

```
import json

try:
    with open("books.json", encoding="utf-8") as file:
        books = json.load(file)
except FileNotFoundError:
    print("Soubor neexistuje.")
except json.JSONDecodeError as error:
    print("Neplatny JSON:", error)
```

Zachytáváme konkrétní chyby, kterým rozumíme. Chybu v dalším zpracování neskrýváme širokým `except Exception`.

Platný JSON ještě nemusí mít správný obsah

Oba texty jsou syntakticky platný JSON:

Očekáváme

seznam knih se jmény polí `title`, `author`, `year`

Můžeme dostat

číslo 42, prázdný objekt nebo knihu bez roku

Po deserializaci kontrolujeme **strukturu, povinná pole, typy a povolené hodnoty**.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

CSV popisuje posloupnost řádků a polí

- První řádek může obsahovat názvy sloupců.
- Každý další řádek představuje jeden záznam.
- Oddělovačem nemusí být vždy čárka; běžný je i středník.
- Uvozovky dovolují uložit oddělovač nebo nový řádek uvnitř pole.

Důležitý důsledek

CSV přirozeně vyjadřuje plochou tabulku, ale samo nenese datové typy ani vztahy mezi tabulkami.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

`DictWriter` mapuje slovník na řádek

```
import csv

fields = ["title", "author", "year", "available"]

with open("books.csv", "w", encoding="utf-8",
          newline="") as file:
    writer = csv.DictWriter(file, fieldnames=fields)
    writer.writeheader()
    writer.writerows(books)
```

`fieldnames` určuje sloupce i jejich pořadí.

DictReader mapuje řádek na slovník

```
import csv

with open("books.csv", encoding="utf-8",
          newline="") as file:
    reader = csv.DictReader(file)
    loaded_books = list(reader)

print(loaded_books[0])
```

Názvy klíčů převezme z hlavičky. Iterátor lze zpracovávat i postupně bez převodu na seznam.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

CSV vrací pole jako řetězce

```
book = loaded_books[0]

print(book["year"], type(book["year"]))
# 1965 <class 'str'>

print(book["available"],
      type(book["available"]))
# True <class 'str'>
```

CSV neví, zda 1965 znamená rok, identifikátor, nebo text. Význam sloupců je součástí naší dohody.

Typy obnovujeme explicitně

```
def parse_bool(text):
    normalized = text.strip().lower()
    if normalized == "true":
        return True
    if normalized == "false":
        return False
    raise ValueError("Expected true or false")

for book in loaded_books:
    book["year"] = int(book["year"])
    book["available"] = parse_bool(book["available"])
```

Převod je dobré soustředit na jedno místo a při neplatném vstupu selhat srozumitelně.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Častá past: `bool("False")` je `True`

```
print(bool(""))        # False
print(bool("False"))   # True
print(bool("0"))       # True
```

Proč?

`bool` u řetězce testuje, zda je neprázdný. Neinterpretuje jeho textový obsah.

Pro CSV proto používáme jasně definovaný převod povolených hodnot.

CSV neskládáme ručně pomocí `split`

```
title,author,note
Duna,Frank Herbert,"science fiction, classic"
```

- Čárka uvnitř poznámky není nový sloupec, protože pole je v uvozovkách.
- Modul `csv` řeší uvozovky, oddělovače i konce řádků.
- Soubor otevíráme s `newline=""`, aby konce řádků spravoval modul.

CSV je jednoduché na pohled, ale má pravidla.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Stejné knihy, jiná dohoda

	JSON	**CSV**
tvar	seznam slovníků	tabulka
čísla a boolean	zachová	načte jako text
vnoření	přirozené	nepohodlné
tabulkový editor	méně pohodlný	přirozený
čtení po řádcích	ne běžný JSON dokument	přirozené
schéma	mimo formát	mimo formát

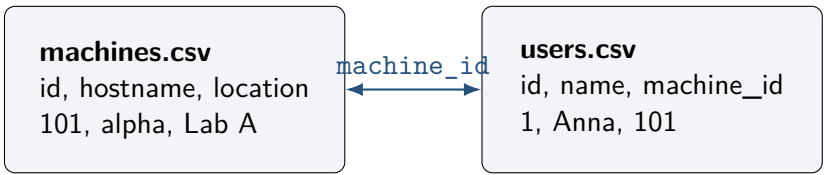
Ani jeden formát není obecně lepší. Každý zpřístupňuje jiné kompromisy.

Vnořená data patří přirozeně do JSONu

```
course = {
    "name": "Programovani",
    "students": [
        {"name": "Anna", "score": 92},
        {"name": "Petr", "score": 71},
    ],
}
```

Objekt obsahuje pole dalších objektů. V JSONu zůstane vztah viditelný přímo ve struktuře.

Tabulky vyjadřují vztah pomocí identifikátoru



Vnořenou strukturu lze rozdělit do více tabulek. Vazbu musíme při načtení znovu spojit a ověřit, že odkazované ID existuje.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Binární formát ukládá bajty, ne čitelný text

- Může být kompaktní nebo rychlý pro konkrétní účel.
- Člověk jej obvykle nezkontroluje v textovém editoru.
- Čtenář musí přesně znát formát a jeho verzi.
- „Binární“ samo o sobě neznamená menší, rychlejší ani bezpečnější.

Příklady

Obrázky, komprimované archivy, databázové soubory a Python `pickle`.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

`pickle` pohodlně uloží Python objekty

```
import pickle

with open("books.pkl", "wb") as file:
    pickle.dump(books, file)

with open("books.pkl", "rb") as file:
    loaded_books = pickle.load(file)
```

Používá binární režimy `"wb"` a `"rb"` a je úzce spojený s Pythonem.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Nedůvěryhodný `pickle` nikdy nenačítáme

Bezpečnostní hranice

`pickle.load` může při načítání spustit libovolný kód. Soubor od cizího člověka, z webu nebo z neověřeného úložiště proto nenačítáme.

- `pickle` není bezpečný výměnný formát.
- Není vhodný pro komunikaci mezi různými jazyky.
- Pro dlouhodobá data může být citlivý na změny tříd a programu.

Pro výměnu běžných dat preferujeme explicitní formát, například JSON.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Velikost a rychlost nejdřív měříme

- Odsazený JSON je větší než kompaktní JSON.
- CSV může být úsporné pro plochá data, protože neopakuje názvy polí.
- Komprese může rozdíl mezi textovým a binárním formátem zásadně změnit.
- Čas často spotřebuje disk, síť nebo další zpracování, ne parser.

Praktické pravidlo

Nejprve volte podle správnosti, přenositelnosti a jednoduchosti. Optimalizujte až podle měření na skutečných datech.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Velká data nechceme vždy načíst celá

CSV	`DictReader` lze procházet řádek po řádku.
JSON	Jeden velký objekt či pole `json.load` běžně načte do paměti celé.
JSON Lines	Každý řádek je samostatný JSON objekt; lze jej zpracovat postupně.

Pozor

JSON Lines je jiná dohoda než jeden běžný JSON dokument. Odesílatel i příjemce ji musí znát.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Rozhodovací mapa

Potřeba	**Rozumný výchozí formát**
jednoduchá tabulka	CSV
otevření v tabulkovém procesoru	CSV
vnořená struktura a běžné typy	JSON
výměna mezi různými jazyky	JSON
postupné zpracování záznamů	CSV nebo JSON Lines
vlastní důvěryhodný Python mezivýsledek	někdy `pickle`
dlouhodobá či důležitá data	dokumentovaný stabilní formát

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Časté chyby

- Přepsání jediného zdrojového souboru režimem `"w"` před ověřením výsledku.
- Záměna `dump`/`dumps` nebo `load`/`loads`.
- Předpoklad, že CSV obnoví čísla a logické hodnoty samo.
- Použití `bool(text)` pro převod textové logické hodnoty.
- Ruční rozdělování CSV pomocí `split(",")`.
- Důvěra v příponu souboru bez ověření obsahu.
- Načtení nedůvěryhodného souboru pomocí `pickle`.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Rychlá kontrola porozumění

1. Jaký je rozdíl mezi serializací a zápisem souboru?
2. Co vrátí JSON po uložení a načtení n-tice?
3. Proč je rok načtený přes `DictReader` řetězec?
4. Proč `bool("False")` nepřevede text podle očekávání?
5. Kdy je přirozenější CSV a kdy JSON?
6. Za jakých podmínek lze bezpečně použít `pickle.load`?

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Shrnutí

- Serializace převádí datový model na text nebo bajty; deserializace vytváří nové objekty.
- JSON zachovává běžné typy a přirozeně vyjadřuje vnoření.
- CSV je přirozená tabulka, ale typy sloupců obnovujeme sami.
- Formát volíme podle tvaru dat, příjemce, velikosti a důvěryhodnosti.
- Platná syntaxe nezaručuje správný obsah; načtená data validujeme.
- Nedůvěryhodný `pickle` nikdy nenačítáme.

Navazující komentované příklady

Porovnáme JSON a CSV nad stejnými daty, obnovíme typy a převedeme tabulku do vnořené struktury.